Berner Fachhochschule
Haute école spécialisée bernoise
Bern University of Applied Sciences



IAM

Authentifizierungsprotokolle - OAuth 2.0

Prof. Dr. Annett Laube, HS 26, HSLU

Open Authorization 2.0 – oAuth 2.0



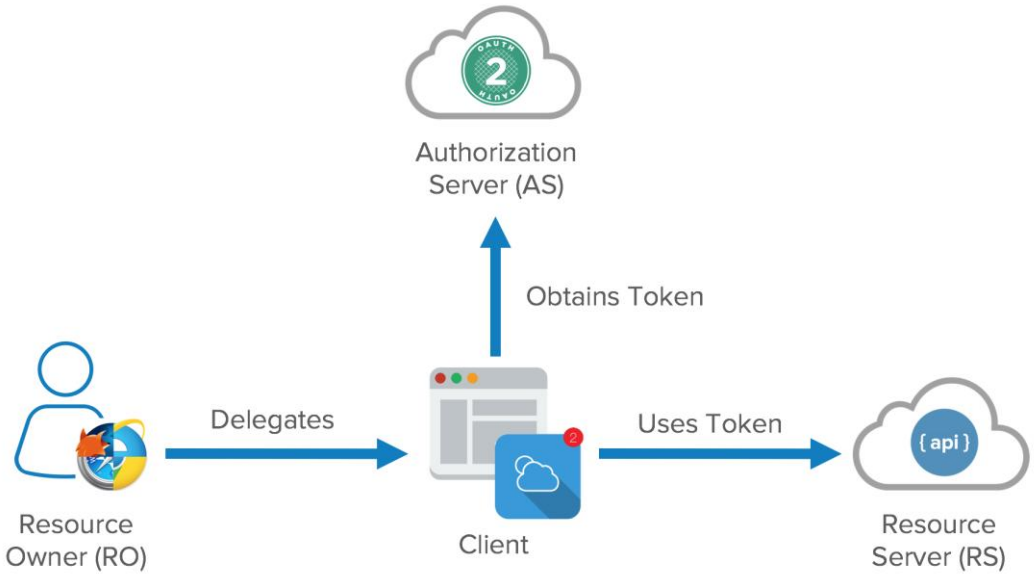
- Offener Standard für (delegierte) **Autorisierung**
  - Ist kein Authentifizierungsprotokoll
- Ermöglicht Geräten, API's, Servern und Applikationen limitierten Zugriff auf geschützte Ressourcen durch sogenannte **Access Tokens**
- Basiert auf standardisierten Nachrichten (JSON über HTTPS)
- Liefert keine Informationen zur Art der Authentifizierung oder über das Subjekt
- Ist die Basis für weitere Protokolle, wie OIDC, UMA 2.0 oder IndieAuth

- Basiert auf RFC 6749 (https://tools.ietf.org/html/rfc6749) und RFC 6750 (https://tools.ietf.org/html/rfc6750)

Problem der Delegierung von Autorisierung

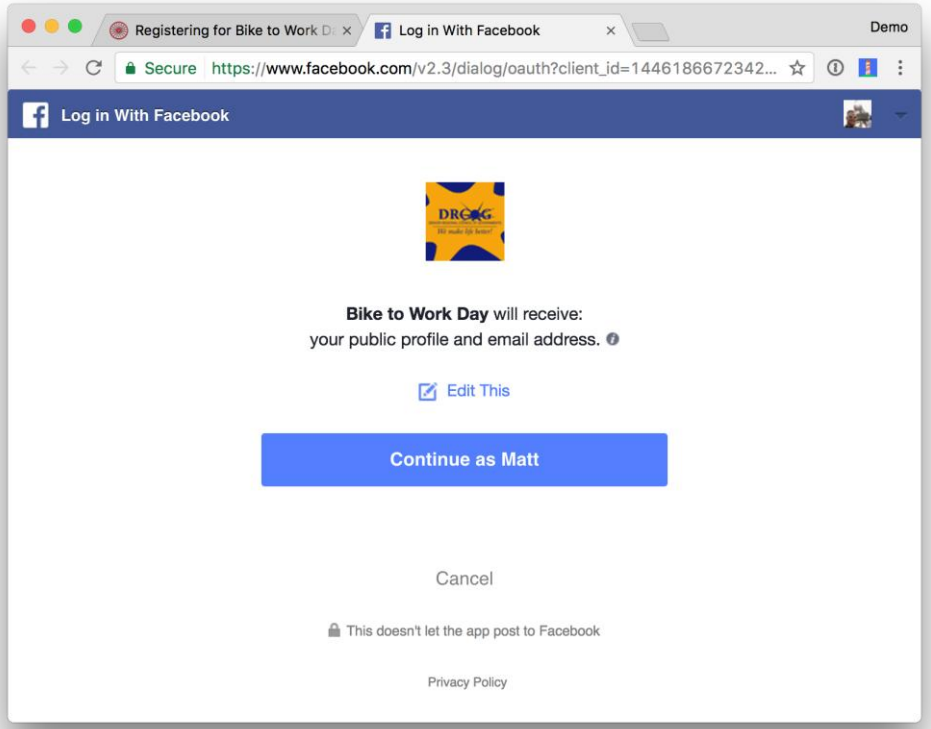
- OAuth ist ein Framework zur Delegierung von Autorisierung. Es ermöglicht Anwendungen einen begrenzten Zugriff (Scopes) auf die Daten eines Nutzers
  - „Wie kann ich einer Anwendung den Zugriff auf meine Daten erlauben, ohne ihr mein Passwort zu geben?“
- Entkoppelung Authentifizierung und Autorisierung

1. Anwendung (Client) fordert Autorisierung vom Benutzer an
2. Benutzer autorisiert Anwendung
3. Anwendung legt dem Server die Autorisierung vor, um ein Token zu erhalten
4. Mit dem Token wird sichergestellt, dass nur diese Anwendung Zugriff auf die vom Benutzer für sie freigegebenen Daten hat



4 Rollen (Actors) (1/3)

- **Resource Owner** (Benutzer/User, Subjekt):
  - Besitzer der Daten (z.B. Facebook-Profil) auf dem **Resource Server**
  - gibt einer Anwendung Zugriff auf seine Daten (Benutzerkonto, Identität), die von einer anderen Anwendung (oft von einem IdP) verwaltet werden.
  - Dabei werden nicht die Authentifizierungsdaten an die Anwendung weitergeben, sondern nur der «scope» der Autorisierung (z.B. Lese- oder Schreibzugriff auf die freigegebenen Daten).

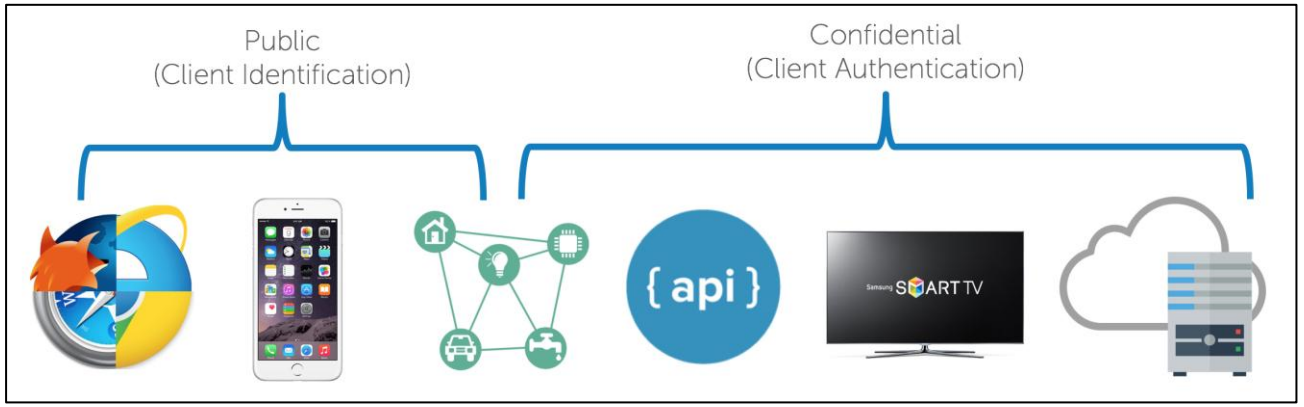


4 Rollen (Actors) (2/3)

Client (Anwendung/Application, RP):

- Der Client ist die Anwendung, die auf die Benutzerdaten zugreifen möchte.
- Dazu muss der Client vom *Resource Owner* entsprechend autorisiert werden.

Public Clients (default)	*Confidential Clients*
▸ Keine vertrauenswürdige Umgebung (Browser, Mobile-App, IoT)	▸ Können «Client Secrets» in einer sicheren Umgebung verwalten (z.B. Smart TV, API, Gerät). ▸ End-User haben keine Zugriffsmöglichkeit auf diese Umgebung.



4 Rollen (Actors) (3/3)

- **Authorization Server** (IdP)
  - Der Authorization Server authentifiziert das Subjekt und stellt ein zeitlich begrenztes Access-Token für einen von ihm definierten Anwendungsbereich (scope) an die Anwendung (Client) aus.
  - Mit dem **Scope** wird genau definiert, welche Art Zugriff auf welche Daten möglich ist (z.B. read/write)

- **Resource Server** (IdP)
  - Der Resource Server hostet und schützt die Benutzerkonten.

- Resource und Authorization Server werden **zusammen** auch als **OAuth-Server** oder **Service API** bezeichnet.

Tokens

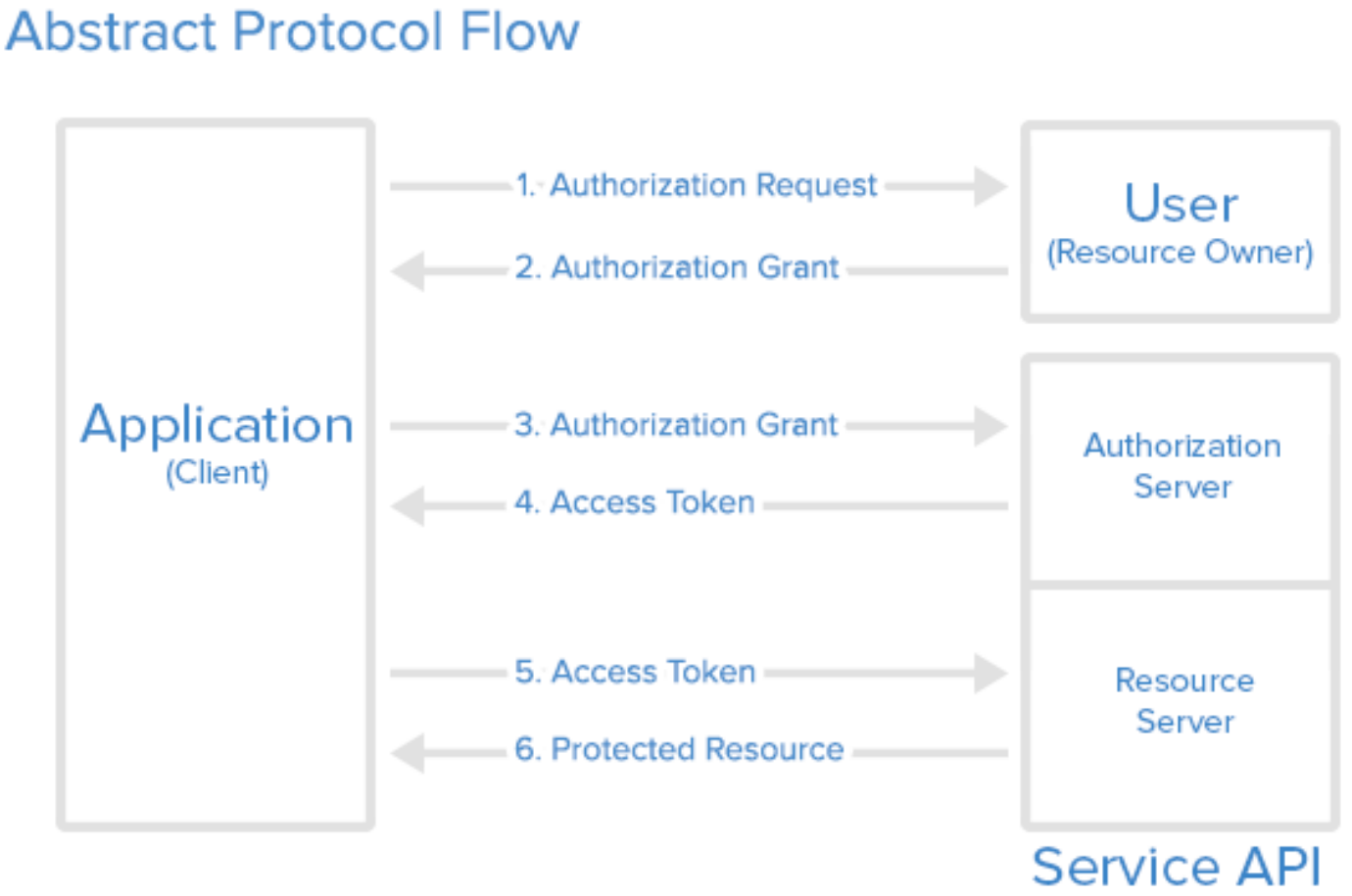
- **Access Token**:
  - Mit dem Access Token erfolgt der Zugriff auf die Ressource. Es ist meist kurzlebig (Stunden oder Minuten).

- **Refresh Token**:
  - Sind langlebiger (Tage, Monate, Jahre) und können revoziert werden.
  - Mit Refresh Token kann man neue Access Token erlangen, ohne dass der Benutzer sich erneut authentifizieren muss.
  - Wird meist zusammen mit dem Access Token ausgestellt.

- OAuth definiert nicht den Inhalt der Token. Meist werden (signierte) JSON Web Token verwendet.

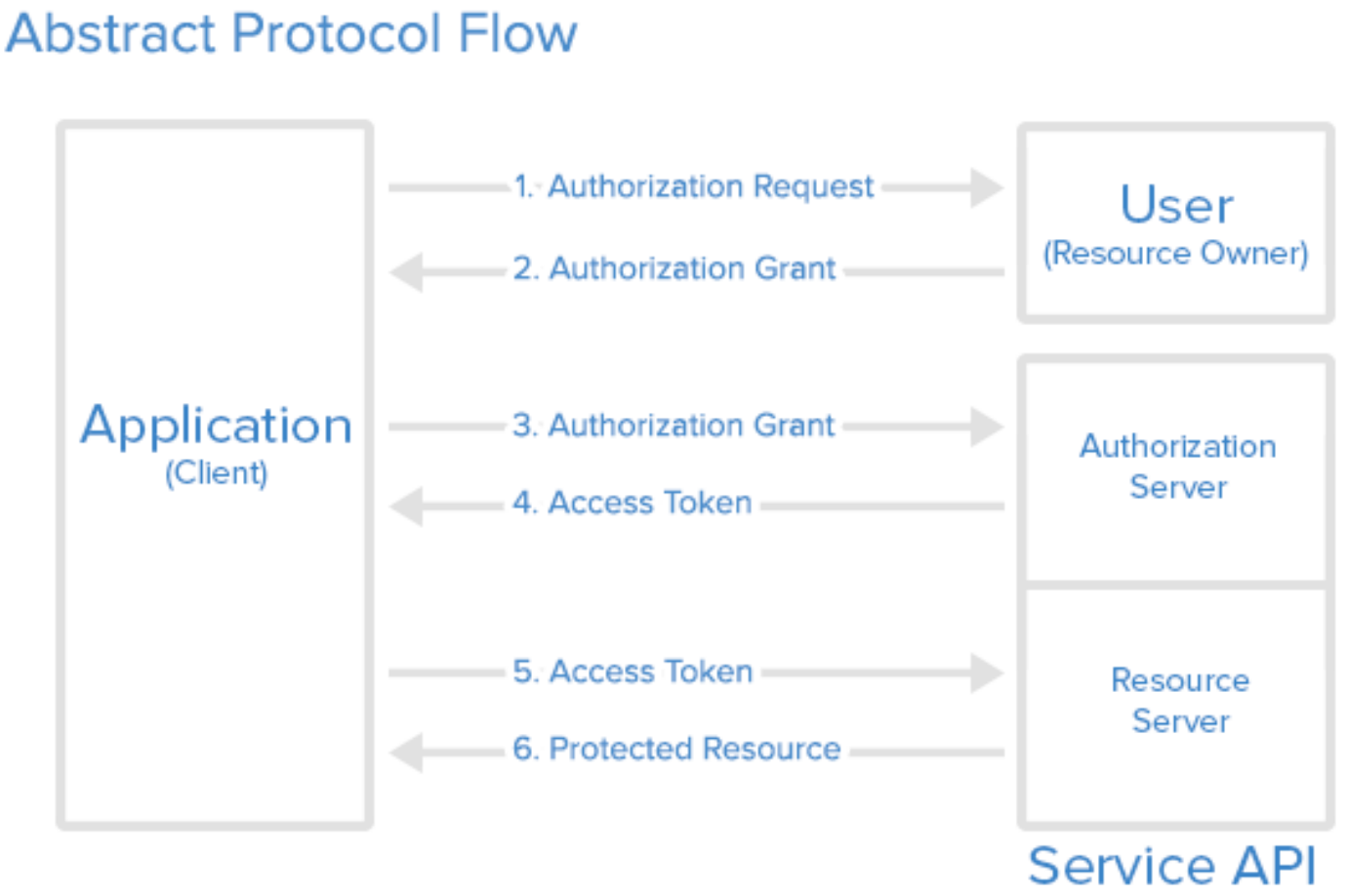
Genereller Protokollfluss (1/2)

1. Die Anwendung fordert vom Benutzer die Autorisierung für den Zugriff sein Benutzerkonto an.
2. Wenn der Benutzer den Request erlaubt, bekommt die Anwendung einen *Authorization grant.*
3. Die Anwendung fordert nun ein *Access token* vom *Authorization Server*. Sie präsentiert dabei ihre eigene Identität und das *Authorization grant.*



Genereller Protokollfluss (2/2)

4. Wenn die Identität der Anwendung verifiziert wurde (auf Basis einer vorhergehenden Registrierung) und der Authorization grant gültig ist, gibt der *Authorization Server* ein *Access Token* für die Anwendung aus. Die Autorisierung ist abgeschlossen.
5. Die Anwendung fordert die Ressource vom *Resource Server* an und präsentiert das Access Token.
6. Wenn das Access Token gültig ist, stellt der *Resource server* die Ressource für die Anwendung bereit.



Diskussion

- Schauen Sie sich den generellen Protokollfluss genau an.
  1. Wann wird der Benutzer authentifiziert?
  2. Wer wird noch authentifiziert?

Registrierung der Anwendung

Jede Anwendung (Client) muss beim Service (Authorization und Resource Server) mit den mindestens den folgenden Informationen registriert werden:

- *Application Name*
- *Application Website*
- *Redirect URI or Callback URL*: wird verwendet, um den Benutzer nach dem Authentifizierungsprozess an die Anwendung zurückzuschicken. Dieser Teil der Anwendung muss mit den Access Tokens umgehen und sie verwenden.

Am Ende der Registrierung, bei der meist auch die Anwendung verschiedenen Test unterzogen wird, bekommt die Anwendung vom Service

- einen *Client Identifie*r (Client ID, App ID): Eindeutiger öffentlicher Name, zur Identifizierung der Anwendung durch den Service
- ein *Client Secret*: wird zur Authentifizierung der Anwendung verwendet (meist eine lange Zufallszahl in Hex); soll geheim gehalten werden.

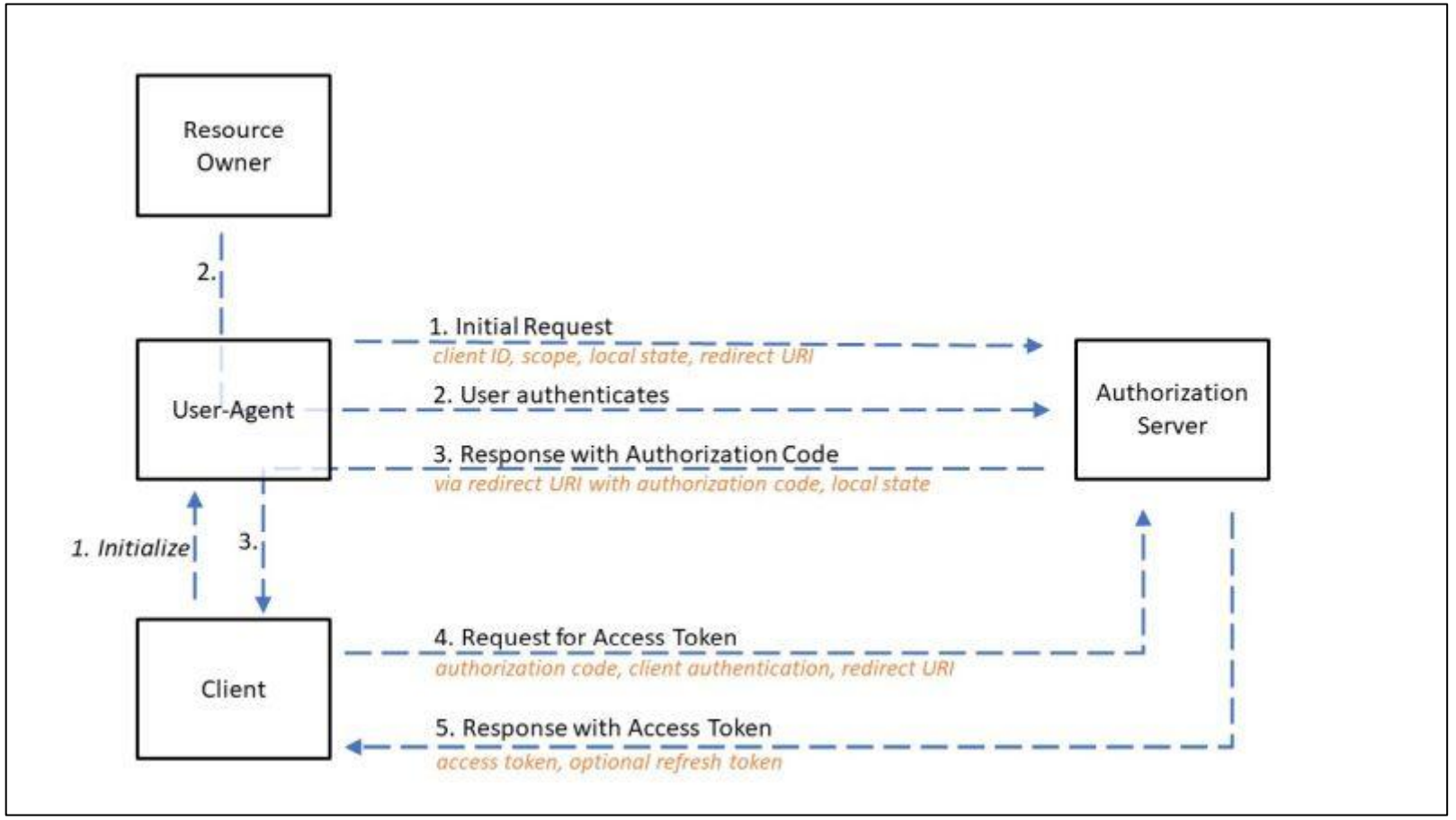
Authorization grants & flows

- Je nach Methode, die die Anwendung verwendet, um eine Autorisation anzufordern, werden 4 verschiedene *Authorization Grants* unterschieden, die dann in verschiedene Protokollabläufe resultieren:
  - **Authorization Code**: bei serverseitigen Anwendungen
  - **(Implicit)**:
    - bei Mobile Apps oder Webanwendungen (Anwendungen, die auf dem Gerät des Benutzers laufen)
    - Wird nicht mehr empfohlen und durch *Authorization code + PKCE* ersetzt
  - **Resource Owner Password Credentials**: für vertrauenswürdigen Anwendungen, wie z.B. denen, die dem Service selbst gehören
  - **Client Credentials**: beim API-Zugriff von Anwendungen. Nur Client-Authentifizierung; Resource Owner ist nicht beteiligt (z.B. zur Inter-Service-Kommunikation)

Authorization Code Flow

`grant_type="authorization_code"`

Resource Owner (user agent) bekommt Authorization Code nach erfolgreicher Authentifizierung (Front Channel), der Client tauscht diesen Code gegen ein Access Token, opt. Refresh Token, ein (Back Channel)

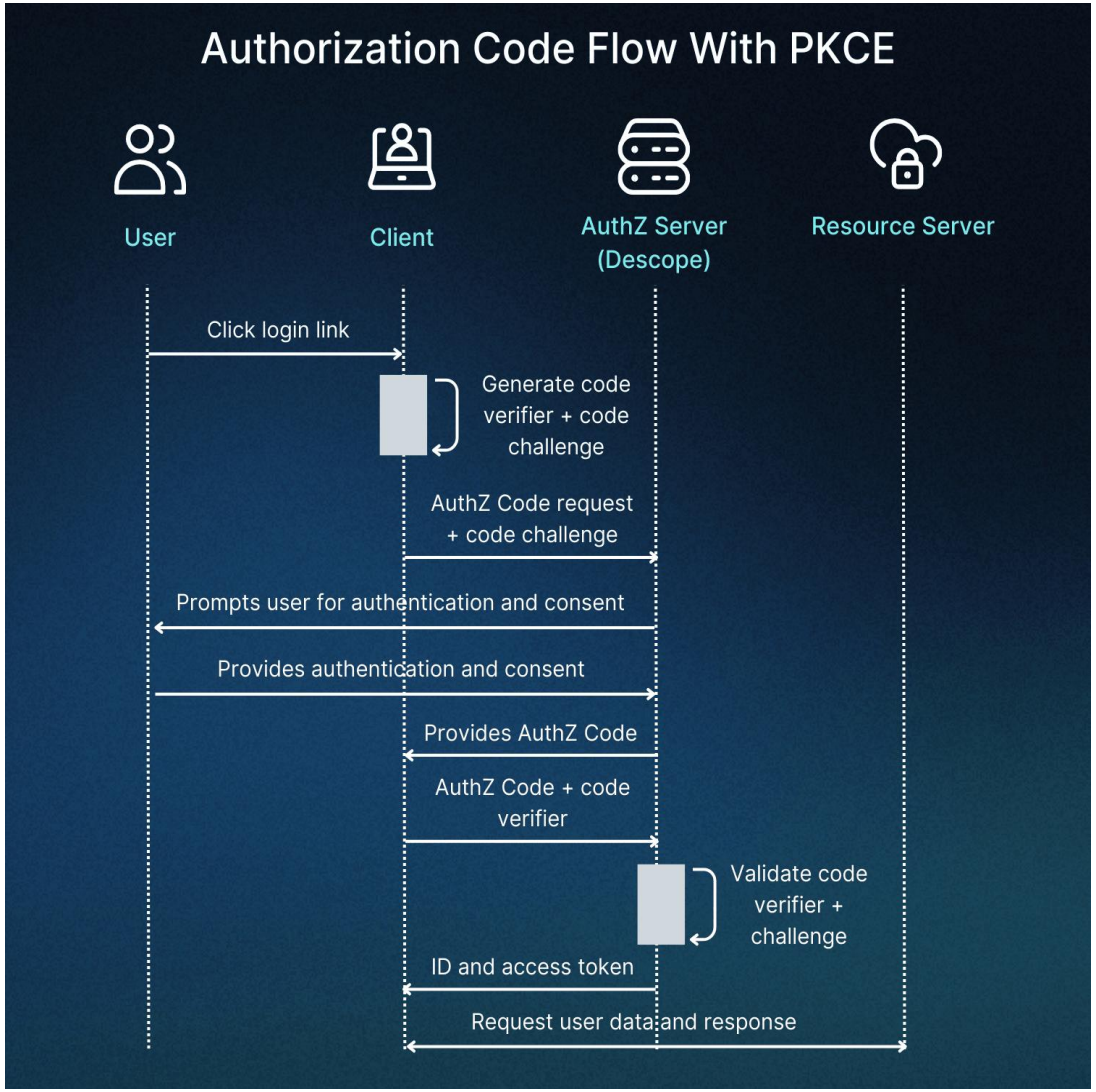


Authorization Code Flow mit PKCE

PKCE = Proof Key for Code Exchange

Erweiterung des Autorisierungscode-Ablaufs zur Abwehr von CSRF- und Autorisierungscode-Injection-Angriffen

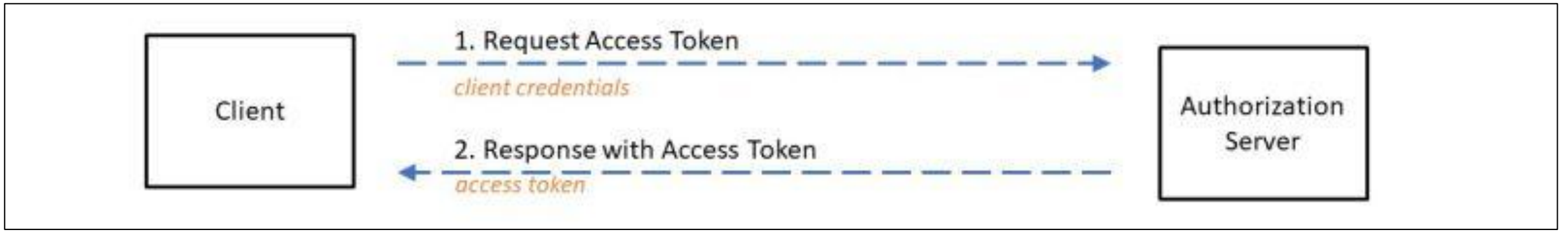
- Vor der Abfrage des Authorization Codes wird ein *Code verifier* (Zufallszahl) erzeugt und gehasht -> *Code challenge*
- Beim Eintauschen des Access Tokens muss nochmal die *Code challenge* mitgegeben werden
  - Auth Server überprüft, dass die *Code challenges* identisch sind (und damit vom gleichen Client kommen)



Client Credentials Flow

```
grant_type="client_credentials"
```

- Nur für vertrauenswürdige Clients
- Wird eingesetzt, wenn der Client der Resource Owner selbst ist oder der Client bereits zuvor autorisiert wurde.
- Für M2M Applikationen, wie zum Beispiel Daemons oder Services im Backend.
- Client verwendet nur seine Zugangsdaten oder andere Formen von Client Authentifizierung gegenüber dem Authorization Server.



Kritische Aspekte

- OAuth fehlen (als Autorisierungsprotokoll) einige für das Login wichtige Fähigkeiten:
  - der OAuth-Autorisierungsserver stellt dem Client (bewusst) keine Benutzer-ID oder andere Benutzerdaten zu Verfügung.
  - Es fehlen Funktionen zur Beeinflussung des Authentifizierungsprozesses (z. B. zum Erzwingen eines Logins) und zur gesicherten Übertragung von Informationen zum Authentifizierungsprozess.

- Dienstanbieter wie Facebook und Amazon haben für den Zweck des Logins proprietäre OAuth-Erweiterungen implementiert, bei dem ein OAuth Access Token verwendet wird, um an die Benutzerdaten zu kommen.

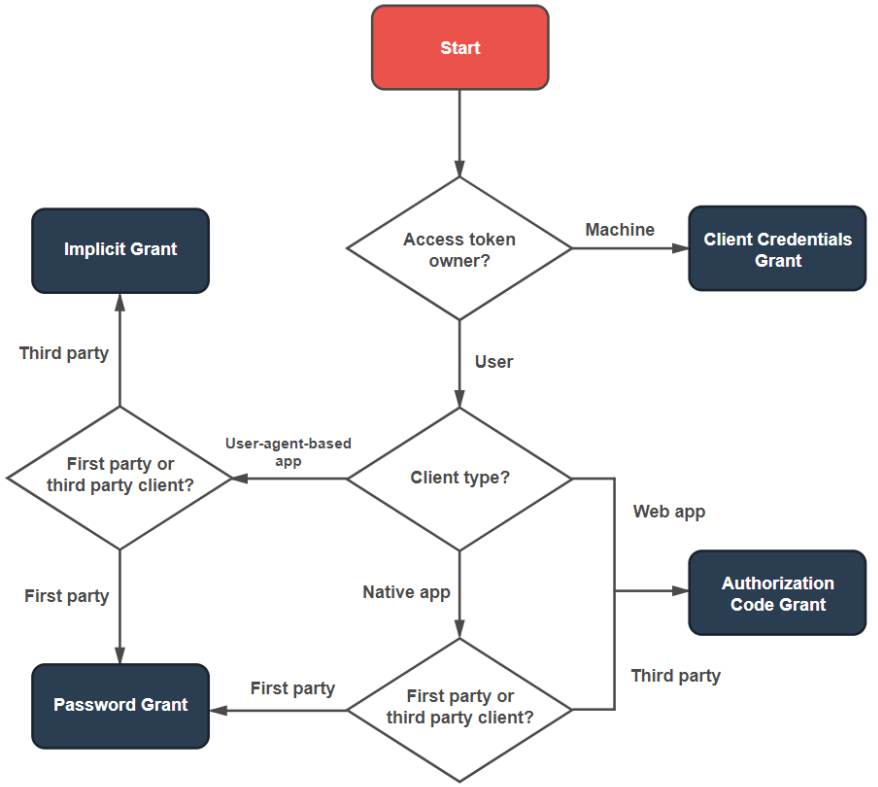
Typische Einsatzgebiete

- Soziale Netzwerke, wie Facebook, Google oder X
  - Login & Teilen von Inhalten
- Cloud-Speicher, Zugriff auf Dateien:
  - Anwendungen können auf Dateien in Cloud-Speichern wie Google Drive, Dropbox oder OneDrive zugreifen, um sie zu bearbeiten oder herunterzuladen.
- Anbindung von Zahlungssystemen in Online-Shops
- **Zugriff auf externe APIs**:
  - Entwickler können OAuth 2.0 verwenden, um auf die APIs anderer Dienste zuzugreifen, z.B. um Wetterdaten abzurufen oder Karten anzuzeigen.
- IoT-Geräte: Fernzugriff und Steuerung

Sicherheitsaspekte

Um OAuth 2.0 sicher zu implementieren, muss

- der richtige Flow gewählt werden,
- Zugriffstoken und Schlüsselmaterial sicher gespeichert werden,
- Gültigkeitsdauer von Authorization Code und Access Token begrenzt werden,
- HTTPS für sichere Kommunikation verwendet werden
- …



OAuth 2.1

OAuth 2.1 hat das Ziel, die am häufigsten genutzten Funktionen von OAuth 2.0 zu konsolidieren und zu vereinfachen.

Die wichtigsten Änderungen:

- PKCE ist für alle OAuth-Clients erforderlich, die den Autorisierungscode-Flow verwenden;
- Weiterleitungs-URIs müssen exakt verglichen werden
- Implicit grant wurde entfernt
- Bearer-Tokens dürfen nicht im Abfrage-String von URIs verwendet werden
- Refresh-Token müssen entweder auf den Absender beschränkt sein oder dürfen nur einmal verwendet werden.
- Die Definitionen von public/confidential clients wurden vereinfacht

Fragen oAuth 2.0 / OIDC

1. Was ist die Hauptfunktion von oAuth 2.0?

2. Welche Akteure gibt in OAuth 2.0 und welche Funktionen haben sie?

3. Wodurch unterscheidet sich OIDC von oAuth 2.0 ?

Quellen

- https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2
- https://www.oose.de/blogpost/oauth-openid-connect-und-jwt-wie-haengt-das-alles-zusammen-teil-1/
- https://developer.okta.com/blog/2017/06/21/what-the-heck-is-oauth
- https://aaronparecki.com/oauth-2-simplified/#authorization

- Datenschutz: Einführung in technischen Datenschutz, Datenschutzrecht und ... By Ronald Petrlic, Christoph Sorge,Springer 2017 https://books.google.ch/books?id=JdiTDgAAQBAJ&lpg=PA6&ots=e9q34OZHf7&dq=oAuth%20OIDC%20einf%C3%BChrung&pg=PA72#v=onepage&q=oAuth%20OIDC%20einf%C3%BChrung&f=false

- FaceBook-Login: https://www.vertabelo.com/blog/how-to-store-authentication-data-in-a-database-part-4-implementing-the-login-with-facebook-button-in-python/